软件需求分析与设计

第四章框架梳理

仅供学习参考

目 录

一、 体系结构优先权（Architectural Prerogatives）- 1 -

（一） 体系结构设计 - 1 -

1. 软件体系结构 - 1 -

（二） 模型—视图—控制器框架（Model—View—Controller，MVC） - 1 -

1. 定义及性质 - 1 -

（三） J2EE 的核心体系结构（Java 2 Platform Enterprise Edition） - 1 -

1. 定义及性质 - 1 -

（四） 表示—控制器—bean—中介者—实体—资源（PCBMER） - 2 -

1. 定义和性质 - 2 -

2. PCBMER 的 6 个层次 - 2 -

3. PCBMER 的 3 个优点 - 3 -

4. PCBMER 的 7 条原则 - 3 -

二、 状态规格说明（State Specifications） - 4 -

（一） 状态（State） - 4 -

1. 对象的状态 - 4 -

2. 状态规格说明 - 4 -

（二） 类建模——迭代增量式 - 4 -

1. 发现类 - 4 -

2. 对类进行说明 - 5 -

（三） 关联（Association）建模 - 5 -

1. 发现关联 - 5 -

2. 说明关联 - 5 -

（四） 聚合（Aggregation）及复合（Composition）关系建模 - 6 -

1. 4 种聚合语义 - 6 -

2. 发现聚合和复合的方法 - 6 -

3. 说明聚合和复合的方法 - 6 -

（五） 泛化（Generalization）关系建模 - 6 -

1. 泛化的 2 个目的 - 6 -

（六） 接口建模 - 7 -

仅供学习参考

1. 接口（interface）..... - 7 -
2. 表示接口的方法..... - 7 -
（七） 对象建模..... - 7 -
1. 对象建模的作用..... - 7 -
三、 行为规格说明（Behavior Specification）..... - 8 -
（一） 用例建模..... - 8 -
1. 用例的本质特性..... - 8 -
（二） 活动建模..... - 8 -
1. 发现动作的方法..... - 8 -
（三） 交互建模..... - 9 -
1. 交互图的分类..... - 9 -
2. 消息的分类..... - 9 -
（四） 操作建模..... - 9 -
1. 发现类的方法..... - 9 -
四、 状态变化规格说明（State Change Specifications）..... - 10 -
（一） 对象状态建模..... - 10 -
1. 性质..... - 10 -

仅供学习参考

一、体系结构优先权（Architectural Prerogatives）

（一）体系结构设计

1. 软件体系结构

(1) 软件体系结构定义了系统中相互作用的软件构件及**子系统**的结构及组织形式

(2) 重点放在**功能性需求**上，非功能性需求暂不考虑

(3) 所有软件建模的最重要的目标就是将**构件依赖最小化**

(4) 目的是开发一个能用好用的软件，利于软件开发

(5) 架构设计：层次化结构

(6) 构建软件的适应性的**重要必要条件**——遵照某个公认的体系结构框架，优先进行体系结构设计

（二）模型—视图—控制器框架（Model—View—Controller，MVC）

1. 定义及性质

(1) MVC 的 3 个**抽象类**对象

① 模型对象：表示数据对象——应用问题域中的业务实体和业务规则

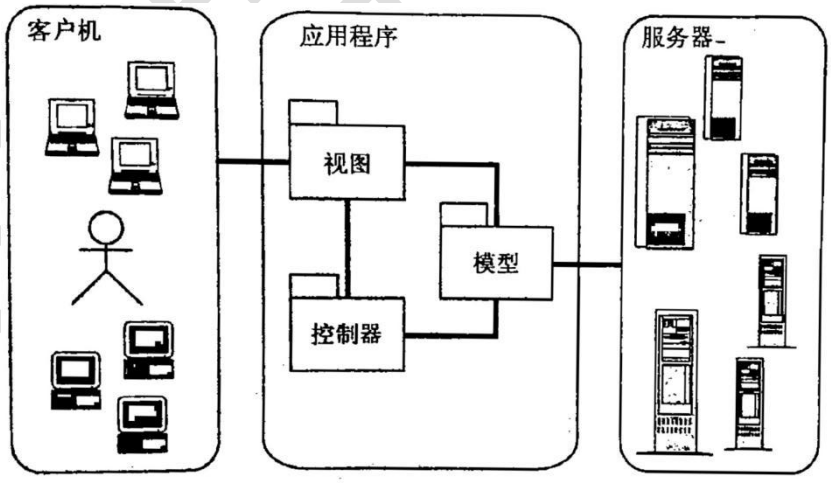
1) 模型对象的变化由**事件处理**通报给视图对象和控制器对象

a. 事件处理：发布者给订阅者对象发送通知的过程（发布者/订阅者技术）

② 视图对象：表示用户界面（UI）对象

③ 控制器对象：表示鼠标和键盘事件

(2) MVC 是**框架、元模型、编程环境**



MVC框架

（三）J2EE 的核心体系结构（Java 2 Platform Enterprise Edition）

1. 定义及性质

(1) J2EE 是以 MVC 为骨架的扩展，用以简化和规范企业应用系统的开发和部署

(2) J2EE 是**分层结构**，表示层、业务层和集成层包含应用程序构件，客户层和 EIS 层处于系统外围

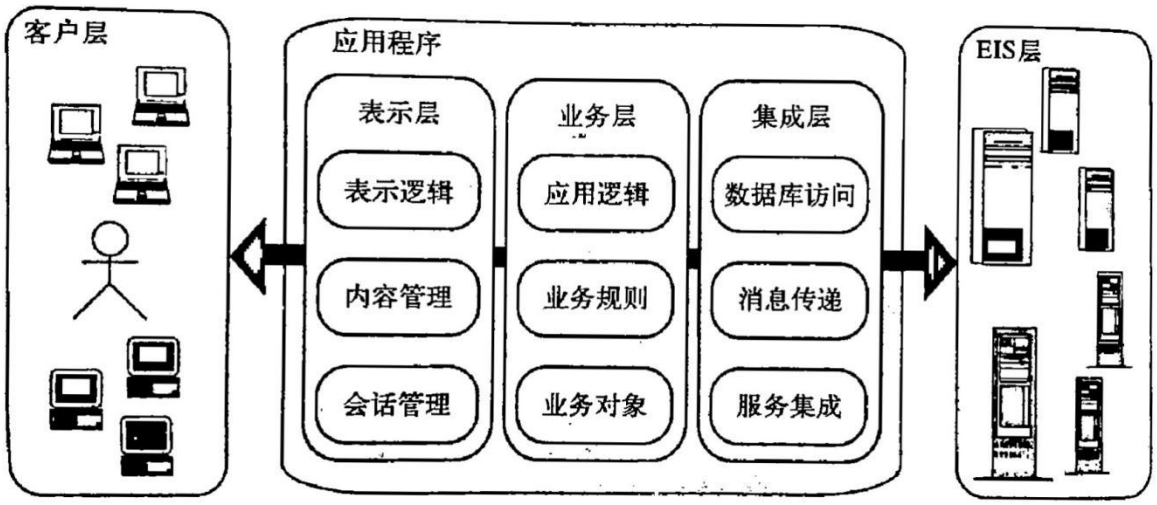
① 客户层——用户通过客户层与系统交互

② EIS 层——企业信息系统层，也称资源层，是任意的持久信息传递系统

③ 表示层——用户通过表示层来访问应用程序

④ 业务层——包含表示层中的控制器构件还没有实现的一部分应用逻辑

⑤ 集成层——负责建立和维护与**数据源**的连接



J2EE的核心框架

（四）表示—控制器—bean—中介者—实体—资源（PCBMER）

1. 定义和性质

(1) PCBMER 体系结构遵循了体系结构设计中广泛认可的发展趋势

(2) 借用了 J2EE 核心框架中的外层（客户层和 EIS 层）

(3) 图中的层表示为 **UML 节点**，带箭头的虚线表示**依赖关系**

(4) PCBMER 的层次**不是**严格线性的，上层拥有的相邻下层可以不止一个

(5) 核心框架的两种版本

① 用 UML 包来定义

② 用 UML 子系统来定义

2. PCBMER 的 6 个层次

(1) 表示层——表示屏幕以及呈现 bean 对象的 UI 对象

(2) 控制器层——表示应用逻辑

(3) **bean 层**——表示那些预先确定要呈现在用户界面上的**数据类**和**值对象**

① bean 数据由用户输入或由**实体对象（实体层）**创建

② bean 层**不依赖于**其他层

(4) 中介者层——建立了充当实体类和资源类媒介的通信管道

① 隔离了实体层和资源层，这样一来，二者的变化能够独立地引入

② 在控制器层和实体/资源层之间充当媒介

(5) 实体层——相应控制器和中介者，由描述“业务对象”的类组成

(6) 资源层——负责所有与外部持久**数据源**（数据库、Web 服务等）的通信和连接

3. PCBMER 的 3 个优点

(1) 层之间相关性的分离

(2) 去除了依赖关系的循环，得到了只存在向下依赖的 6 层结构

(3) 确保相当高的稳定性

4. PCBMER 的 7 条原则

(1) 向下依赖原则（Downward Dependency Principle，DDP）——底层比高层更稳定，使通信变为单向

(2) 向上通知原则（Upward Notification Principle，UNP）——使通信变为单向

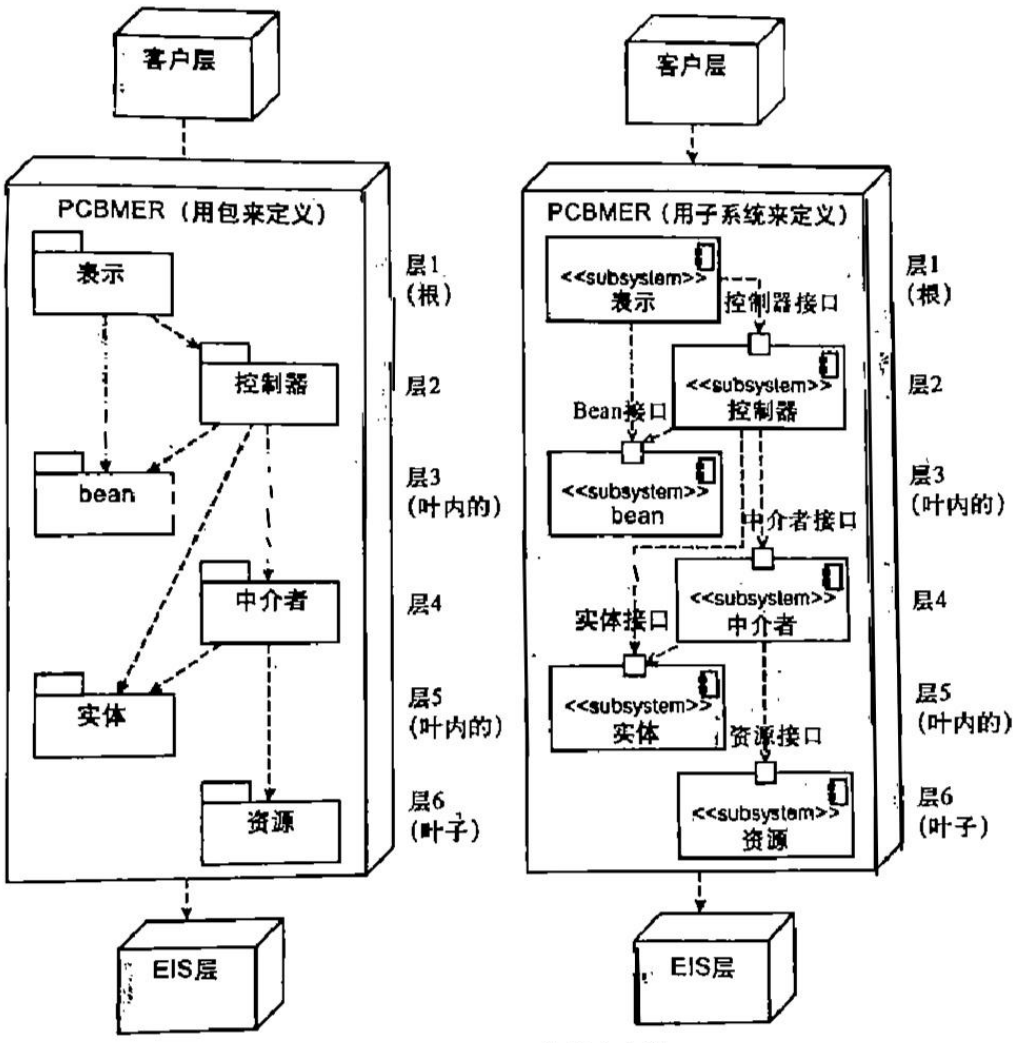
(3) 相邻通信原则（Neighbor Communication Principle，NCP）——使通信只能在相邻节点之间

(4) 显式关联原则（Explicit Association Principle，EAP）——运行时的关联都要在编译时显式出现

(5) 循环去除原则（Cycle Elimination Principle，CEP）——降低系统的复杂性

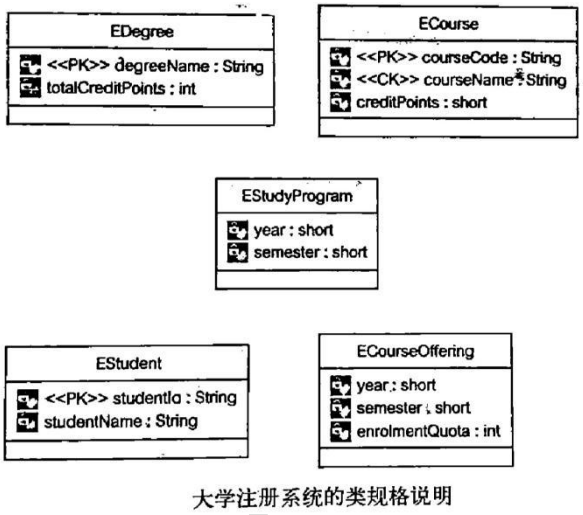
(6) 类命名原则（Class Naming Principle，CNP）——类名 = 层名首字母 + 类名

(7) 相识包原则（Acquaintance Package Principle，APP）——相关类以包的形式封装



PCBMER的核心框架

二、状态规格说明（State Specifications）



大学注册系统的类规格说明

（一）状态（State）

1. 对象的状态

(1) 对象的状态由它的**属性值**和**关联**决定

2. 状态规格说明

(1) 状态规格说明 = **数据结构**模型

(2) 提供系统的**静态**视图

(3) 识别系统中所有类及类与类之间的关系

(4) 侧重于实体类（业务对象）的定义和关系，不考虑类的操作

（二）类建模——迭代增量式

1. 发现类

(1) 名词短语方法（Noun Phrase）——**核心方法**

① 候选类

1) 相关类（Relevant）——类名的名词在需求文档中频繁出现

2) 无关类（Irrelevant）

3) 模糊类（Fuzzy）——最大的问题，需要进一步分析

② 需要假定需求文档是完整而正确的（而事实上很难做到）

(2) 公共类模式方法（Common Class Pattern）——将对象世界划分成有用的组

① 候选类

1) 概念类（如航班预定）

2) 事件类（如航班到达）

3) 组织类（如旅行社）

4) 人员类（如乘客）

5) 地点类（如旅行社办公室）

② 为识别类提供了有用的**指南**，但仅仅只是个指南，不能提供全面、关系紧凑的解决方案

③ 可能导致类名的**误解**

(3) 用例驱动方法（Use Case Driven）

① 有**自底向上**的特点

② 与名词短语方法类似，效率更高一些

③ 准确性依赖于用例模型的完整性和准确性

(4) CRC 方法（类—职责—协作）

① 通过分析对象之间为了完成处理任务而传递的消息来识别类

仅供学习参考

② 作用：一种发现类的技术

③ 重点：如何在系统中均衡分布处理能力

④ 适用于：验证由其他方法发现的类；确定类的属性

(5) 混合方法

① 根据领域知识来发现类的初始集合

② 公共类模式方法提供辅助的指导

③ 名词短语方法和添加新类

④ 用例驱动方法验证已有的类

⑤ CRC 方法对所有类进行集体讨论（头脑风暴）

(6) 发现（实体）类的指南

① 每个类的清晰的目的陈述

② 一组对象的模板描述（一般不考虑单例类）

③ 每个实体类都应该有一组属性

1) 标识属性：主键（keys）

2) 标识类的对象：对象标识符（OID）

④ 每个类应该有区别于其他类的属性

⑤ 类拥有一组操作

2. 对类进行说明

(1) 类的命名

① 每个类都要有一个名字，且互不重名

② 使用单数名词，每个实词首字母大写

③ 在不隐晦、不冗余的前提下使名字尽可能长（<=30）

(2) 类的属性

① 属性名使用小写字母（如 streetName 或 street_name）

（三）关联（Association）建模

1. 发现关联

(1) 主要关联的发现是类发现过程的一个副产品（Side effect）

(2) 类的一些属性就是与其他类的关联

(3) 用例的“预演”（Dry-run）过程可以发现其余的关联

(4) 避免三元及以上的关联

2. 说明关联

(1) 关联的命名

① 一般情况下可以省略，当两个类之间超过一个关联同时存在时，必须有关联名

② 命名规则遵循类名的约定

(2) 关联角色的命名

① 一般省略，用于解释较复杂的关联，特别是自身关联（递归关联）

② 命名规则遵循属性名的约定，并以 the 开头

(3) 确定关联的多重性

① 如果多重性不明确，其上下边界可以省略

（四）聚合（Aggregation）及复合（Composition）关系建模

1. 4 种聚合语义

(1) ExclusiveOwns 聚合——如“书具有章节”的关系

① 存在依赖性（Existence-dependency）：一个复合对象被删除，其构件也不复存在

② 传递性（Transitivity）：C 是 B 的一部分，B 是 A 的一部分，则 C 是 A 的一部分

③ 非对称性（Asymmetricity），A 是 B 的一部分，但 B 不是 A 的一部分

④ 固定属性（Fixed property），B_1 是 A_1 的一部分，则 B_1 决不是 A_i （$i \neq 1$）的一部分

(2) Owns 聚合——如车有轮胎，轮胎依赖于车存在，也可以独立存在，但单独一个轮胎意义不大

① 存在依赖性

② 传递性

③ 非对称性

(3) Has 聚合——两者之间没有存在依赖，如“学院有系，没有某个系也可以存在”的关系

① 传递性

② 非对称性

(4) Member 聚合——有成员关系就行，可以是多对多的多重性，如“会议有主持人”的关系

2. 发现聚合和复合的方法

(1) 在说明这种关系时使用短语**“具有”（has）**和**“是…的组成部分”（is part of）**

(2) 复合是聚合的一种特殊情况

3. 说明聚合和复合的方法

(1) 实心菱形 ◆：复合

(2) 空心菱形 ◇：聚合

（五）泛化（Generalization）关系建模

1. 泛化的 2 个目的

(1) 可替换性（Substitutability）——子类对象可以替换父类变量

(2) 多态性（Polymorphism）——同一个操作在不同子类里有不同的表现形式

2. 发现泛化的方法

(1) 在说明这种关系时使用短语**“可以是”（can be）**和**“是一种”（is a kind of）**

仅供学习参考

3. 说明泛化的方法

(1) 一条带空心箭头的实线，由子类指向父类

（六）接口建模

1. 接口（interface）

(1) 接口**没有属性**（除常量外）、关联或者状态，只有公共的、抽象的**操作**

(2) 接口与类没有关联

(3) 接口可以与另一个接口具有**泛化**关系（一个接口继承另一个接口）

2. 表示接口的方法

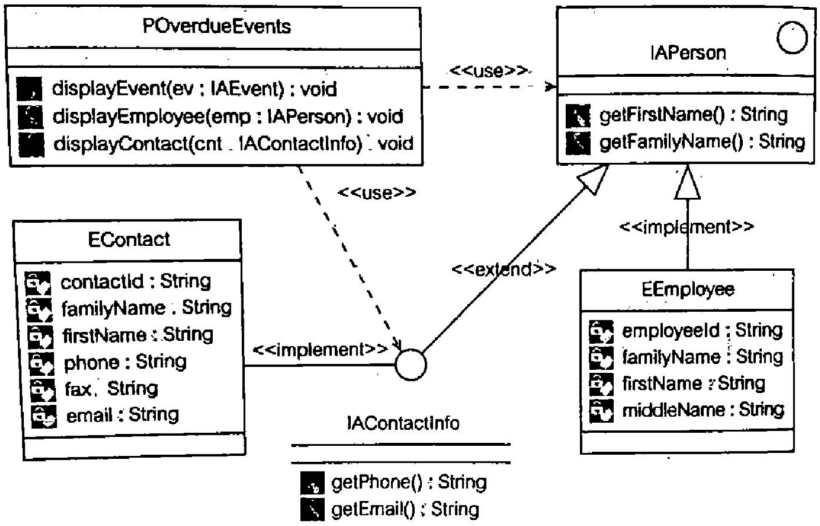
(1) 使用象征类的矩形加上关键字**<<interface>>**

(2) 在矩形的右上角画一个小圆圈

(3) 单独一个圆圈，圆圈下方写上接口名字

(4) 类使用接口时用指向接口的一条虚线箭头表示 - - ->，箭头上标有**<<use>>**

(5) 类实现接口时用一条末端带三角形的虚线表示 - - -▷，虚线上标有**<<implement>>**

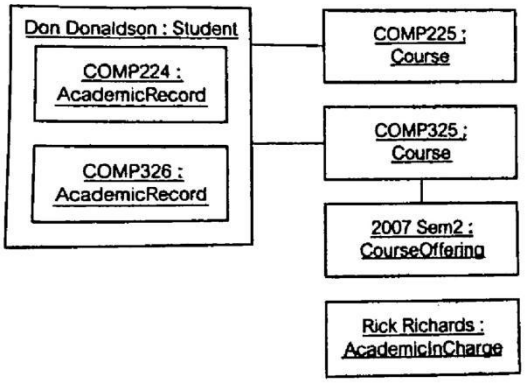


关系管理系统的接口规格说明

（七）对象建模

1. 对象建模的作用

(1) **举例**描述复杂的类和类之间的关系，用对象的例子（**对象图**）来阐明



大学注册系统的对象图

三、行为规格说明（Behavior Specification）

（一）用例建模

1. 用例的本质特性

(1) 一个**完整**的功能

(2) 一个**外部可见**的功能

(3) 一个**正交**的功能

① 正交：用例和用例之间没有冲突、覆盖、交叉和遗漏（不重不漏

② 通过需求依赖矩阵实现

(4) 由一个参与者**启动**的一个功能

(5) 给参与者传递确切值的一个功能（**有返回值**）

2. 发现用例的方法

(1) 需求文档种标识的需求

(2) 系统的参与者以及他们的使用目的

3. 说明用例的方法

(1) 用例图

① 参与者

② 用例

③ 关系

1) 关联：Actor ↔ use_case，表示方法：→或↔

2) 包含：use_case ↔ use_case，表示方法：A—**<<include>>**→B，A 执行 B **一定**执行

3) 扩展：use_case ↔ use_case，表示方法：A—**<<extend>>**→B，A 执行 B **不一定**执行

4) 泛化：Actor ↔ Actor，表示方法：—▷，A—▷B，A 是 B 的一种

5) 依赖：Actor ↔ Actor，表示方法：A - - - **<<depends on>>** - - -> B，A 依赖于 B

（二）活动建模

1. 发现动作的方法

(1) 对描述用例的叙述性规格说明中的语句进行分析，任何带动作的短语都是候选动作

(2) 每个用例都可以有一个或多个活动图建模

2. 说明动作的方法

(1) 活动图标识面向对象程序种的**逻辑流**（顺序流、并发流）

(2) 活动图的要素

① 同步线段 ▌：**并发**线程的发起（分叉）和汇合

② 分支菱形 ◇：**可选**线程的创建（分支）和合并

仅供学习参考

（三）交互建模

1. 交互图的分类

(1) 顺序图：按**时间顺序**展示对象之间的消息交换，在分析中更有用

① 集中式（centralized）：由一个类主要负责（中心控制点），设计简单，但容易出现单点故障

② 分布式（distributed）：没有中心控制点，处理能力分散给多个对象，推荐采用

(2) 通信图：强调**消息交换**时对象之间的关系，在设计中更有用

2. 消息的分类

(1) 信号（Signal）：表示对象间的**异步**通信，发送者在发送信号消息后可以立即继续执行，隐含**事件处理**

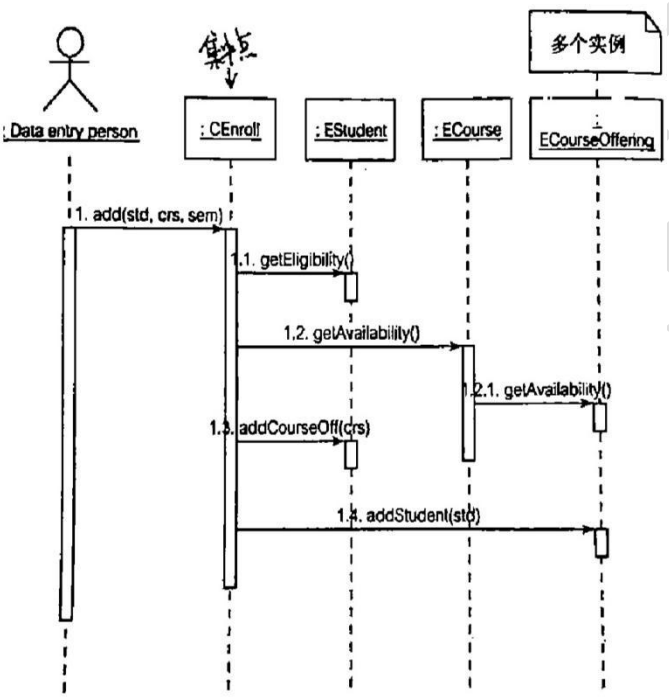
(2) 调用（Call）：表示**同步**的操作调用，需要将控制返回给发送者（有返回值的方法），隐含**消息传递**

（四）操作建模

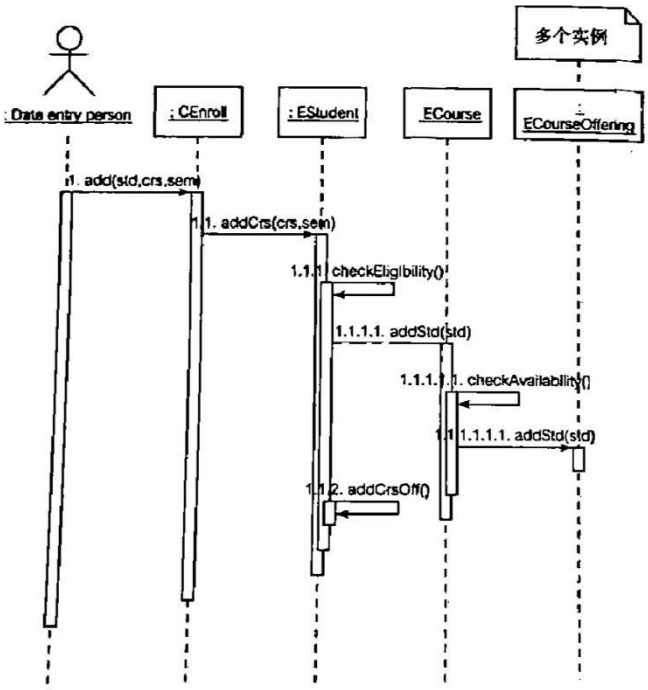
1. 发现类的方法

(1) 从顺序图中发现，每一条消息对应于类图中的方法

(2) 从类或对象的基本功能的方法 如增删改查（**CRUD**）



大学注册系统的集中式顺序图



大学注册系统的分布式顺序图

四、状态变化规格说明（State Change Specifications）

（一）对象状态建模

1. 性质

(1) 应用于工程系统或实时系统

(2) 状态规格说明定义类的**属性**，行为规格说明定义类的**操作**

(3) 在需求阶段的后期或者分析阶段将要结束时进行，主要是对异常的处理

(4) 工具：状态机图

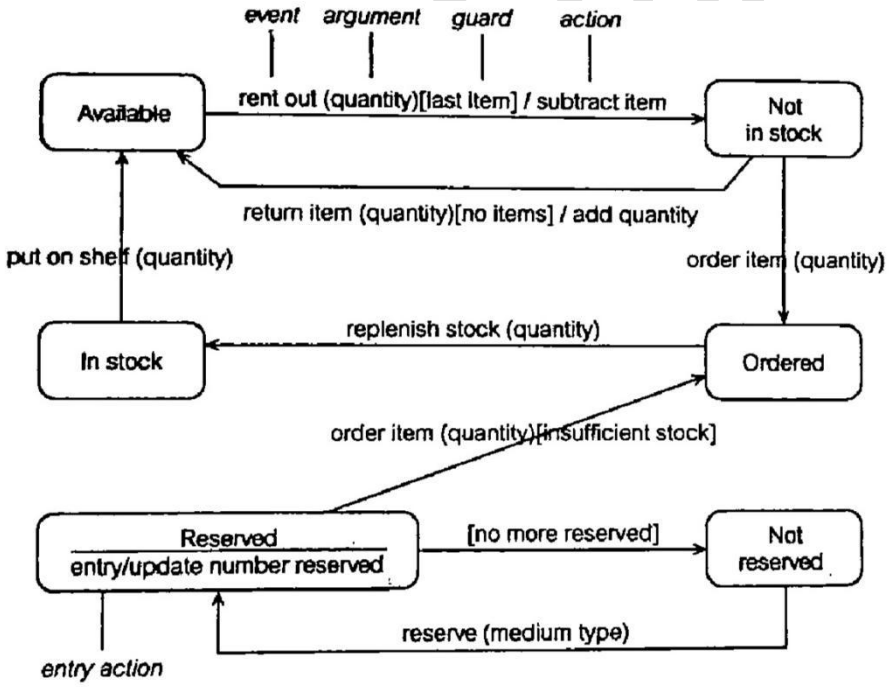
2. 状态转换的触发（trigger）

(1) 信号事件：建立对象间显式异步单向通信

(2) 调用事件：建立同步通信，调用者等待响应

(3) 修改事件：改变守卫值使守卫条件达到**满足**的事件

(4) 时间事件：特定状态下对象满足了一个时间表达式



音像商店系统中的类EMovie的状态机图